01 / HASZNÁLAT

Grover-szimulátor

Szélyes Dávid és tamas722 • Felhasználói és műszaki dokumentáció

Indítás Windows alatt

Csomagold ki a beadási ZIP-et, majd nyisd meg a **GroverSzimulator.html** fájlt a böngészőben. Ez az önálló változat Windowson telepítés és internetkapcsolat nélkül fut. A Pythonból előállított állapotképek, a felület és a grafikonok mind ebben az egy fájlban vannak.

A böngészős változat a **python build_browser.py** paranccsal állítható elő Python 3.10 vagy újabb verzióval, külső csomag nélkül. A grafikonokat a böngésző SVG-vel rajzolja. Az asztali fejlesztői nézet **python app.py** paranccsal fut, a Python tkinter és ttk moduljaira épül.

Egy futás beállítása

Qubitek (1–8) határozza meg az $N = 2^n$ elemszámot. A **Célindex** egész szám 0 és $N - 1$ között. Az **Iterációk** mezőben 0–40 teljes orákulum–diffúzió pár adható meg. Az **Ajánlott iteráció** kitölti a mezőt az első maximumhoz közeli lépésszámmal.

Az **Új futás** gomb a beállításokat ellenőrzi és a szuperpozícióra állítja vissza a nézeteket. Hibás bemenetnél a felület megmutatja a hibát, az előző futás pedig továbbra is bejárható. Az **Előző** és **Következő** gomb vagy a bal és jobb nyíl léptet. Egy teljes iteráció két lépés.

A két ábra olvasása

Balra a bázisállapotok **előjeles amplitúdója** látszik. Arany jelöli a célt, a fehér vonás az előző értéket. Jobbra a folytonos nyíl az aktuális állapot, a szaggatott nyíl az előző állapot. A fejlécben lévő **P(cél)** a célállapot mérési valószínűsége.

02 / MODELL

Mit számol a program?

Egyetlen megjelölt elem, egyenletes kezdőállapot, ideális műveletek

Három állapotkép

Szuperpozíció. Az N bázisállapot kezdeti amplitúdója $1/\sqrt{N}$. Ez az n darab Hadamard-kapu eredménye, ha a regiszter a nullállapotból indul.

Orákulum. A keresett állapot amplitúdóját -1-gyel szorozzuk. Minden más amplitúdó változatlan marad. A mérési valószínűség az amplitúdó négyzete, ezért ez a művelet önmagában csak az előjelet változtatja meg.

Diffúzió. Kiszámítjuk az orákulum utáni amplitúdók m átlagát. Minden a amplitúdó új értéke $2m - a$. Ez az átlag körüli tükrözés, amely a céltól függően növelheti vagy csökkentheti a cél mérési valószínűségét.

A geometriai sík

Az x tengely egységvektora a nem megjelölt állapotok normalizált, egyenletes szuperpozíciója. Az y tengely a célállapot. A cél amplitúdója b, a többi állapot közös amplitúdója a, így $\mathbf{x = \sqrt{(N - 1)} \cdot a}$ és $\mathbf{y = b}$. A koordináták előjelesek, $P(\text{cél}) = y^2$.

Az orákulum az x tengelyre tükröz. A diffúzió a kezdőállapot irányára tükröz. Ha $\theta = \arcsin(1/\sqrt{N})$, a kezdővektor szöge θ, és minden teljes iteráció után 2θ-val nő. A k. teljes iteráció után a cél valószínűsége $\sin^2((2k + 1)\theta)$. Az első csúcs közelébe a $\lfloor \pi/(4\theta) \rfloor$ iteráció visz.

Megvalósítás

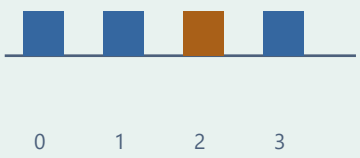
A **grover.py** tárolja a számítási lépéseket. A cél előjelváltása után a diffúzióhoz elég végigmenni a vektoron, ezért egy iteráció számítási ideje $O(N)$. A teljes bejárható előzmény tárolása $O(Nk)$ $k \geq 1$ esetén. A böngésző ugyanabból a Python által számolt állapotképből rajzolja mindkét ábrát. Qubitszámonként elég egy futást tárolni, mert a célindex módosítása csak az amplitúdók helyét cseréli fel.

03 / ELLENŐRZÉS

Két qubit, egy cél

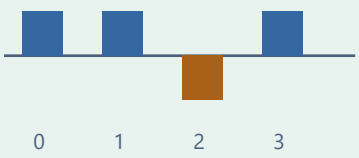
A 2-es célindex és egy teljes iteráció ellenőrizhető kézzel is

Szuperpozíció



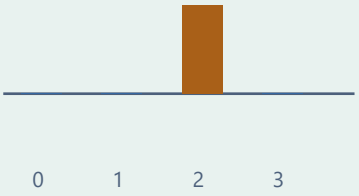
P(cél) = 25%

Orákulum



P(cél) = 25%

Diffúzió



P(cél) = 100%

A cél amplitúdója +0,5 → −0,5 → +1. Az orákulum után az amplitúdók átlaga 0,25. A diffúzió ezért a három nem célállapot amplitúdóját 0-ra viszi, a célét pedig 1-re. A geometriai ábrán x előbb √3/2 marad, majd nullára csökken, míg y követi a cél amplitúdóját.

További próbák

Túlforgatás: állíts be három iterációt. Az első diffúzió után 100%, a második után 25% lesz a cél valószínűsége. **Nulla iteráció:** csak az egyenletes kezdőállapot jelenik meg. **Hibás cél:** két qubit mellett a 4-es indexet a program elutasítja.

Az ellenőrzés 1–8 qubitnél, több célindexre és 40 iterációig hasonlította össze mindkét előjeles koordinátát az elméleti szögekkel. A normát és a bemeneti határokat is ellenőriztük. A program ideális, zajmentes állapotot szimulál, és a cél valószínűségét mutatja. A kvantumalgoritmus lekérdezési előnye nem a klasszikus szimulátor futásidejére vonatkozik.

Források és eszközök

Lov K. Grover, Quantum Mechanics Helps in Searching for a Needle in a Haystack, Phys. Rev. Lett. 79, 325–328 (1997). doi:10.1103/PhysRevLett.79.325
Kvantuminformatikai alkalmazások, 2026 ősz, házi feladatkiírás, 3. feladat.

A 2025-ös C++ beadás a nézetek és a feladat értelmezéséhez szolgált példaként. A Python megvalósításhoz és a dokumentációhoz AI-asszisztenst használtunk.